Reuse or Reload: A GPU-Utilization Study of Batch LLM Inference on Flyte v2

Union.ai / Flyte · batch_inference_saturate.py · Qwen2.5-7B on NVIDIA L4

Abstract—Serving large-model batch inference under an orchestrator forces a choice: keep a GPU worker *warm* across many calls, or spin a fresh container per call. We run one identical vLLM workload — 500 GSM8K questions through Qwen2.5-7B on an L4 — two ways. With container **reuse** on a managed Union cluster the job finishes in **404 s**; a **no-reuse** variant on open-source **Flyte** takes **1,665 s** — **4.1×** slower — and, measured with `nvidia-smi`, keeps the GPU actually computing only **16%** of the run while a model sits resident **86%** of it — where the **reuse** GPU, warm, holds a steady **100%**.

1. Introduction

A 7B model takes minutes to make useful on a GPU: download ~14 GiB of weights, load them, run `torch.compile`, and capture CUDA graphs. That cost is fixed per *process*, not per request. An orchestrator that gives every task its own short-lived container therefore pays it again and again, and the GPU — the expensive resource — spends most of its life idle behind that setup. Persistent *reuse* amortizes the cost across calls and lets a batcher pack work from many concurrent callers into the accelerator. We quantify the gap on a single realistic job.

Table 1. Same workload, two execution models. Union figures in gold, Flyte in purple.

Metric	Union · reuse	Flyte · no-reuse
Wall-clock, 500 questions	404 s	1,665 s (4.1×)
Model loads for the job	2	10
Cold start, amortized / question	~0.5 s	~3.7 s
Effective batch size	up to 256	50
Peak decode throughput	1,424 tok/s	433 tok/s
Mean GPU utilization*	35.4%	15.7%
Utilization while computing	sustained 100%	33-s bursts
Time computing (util > 50%)	35%	16.1%
Time holding a model, idle	61%	86.5%
Per-worker cold start	once	3 m 11 s × 10

2. Experimental Setup

Both runs execute the same example workload (`batch_inference_saturate.py`): Qwen/Qwen2.5-7B-Instruct under vLLM (`gpu_memory_utilization=0.9`, `max_model_len=4096`), 500 GSM8K questions split into ten 50-question chunks, each chunk dispatched as an `infer_batch` call. The **reuse** configuration keeps the code's `ReusePolicy(replicas=2, concurrency=10)`: two persistent L4 workers, each loading the model once and sharing it — via a process-level `TokenBatcher` — across up to ten concurrent calls. The **no-reuse** variant instead runs each chunk in its own fresh pod, cold-loading the model per call.

Environments differ as the platforms do. **Union** built the image with its managed remote builder (vLLM 0.25.1, cu129) and scheduled it on managed GPU nodes. **Flyte** ran on two stock `g6.xlarge` L4 nodes (driver 570.195 / CUDA 12.8) with a hand-built image (vLLM 0.11.0, cu128); utilization was sampled on-node by an `nvidia-smi` DaemonSet every 2 s.

3. Results

Reuse wins on every axis (Table 1). Loading the model twice instead of ten times, and packing concurrent callers into 256-prompt batches rather than isolated 50-prompt calls, yields **3.3×** the peak decode throughput and completes the job **4.1×** faster. The accelerator itself tells the sharper story (Figure 1). Under **Union** reuse (a) the model loads once and the GPU then holds a steady **100%** through batched inference; mean utilization over the whole run is **35%**, and it rises toward 100% as more work amortizes that single load. Under **Flyte** no-reuse (b) the trace *sawtooths*: each chunk loads its own copy of the weights, fires one ~33-s 100% burst, then releases the GPU — ten loads for ten chunks. Integrated, Flyte computes for only **16%** of the run while holding a model resident **86%** of it: almost always *occupied*, almost never *working*.

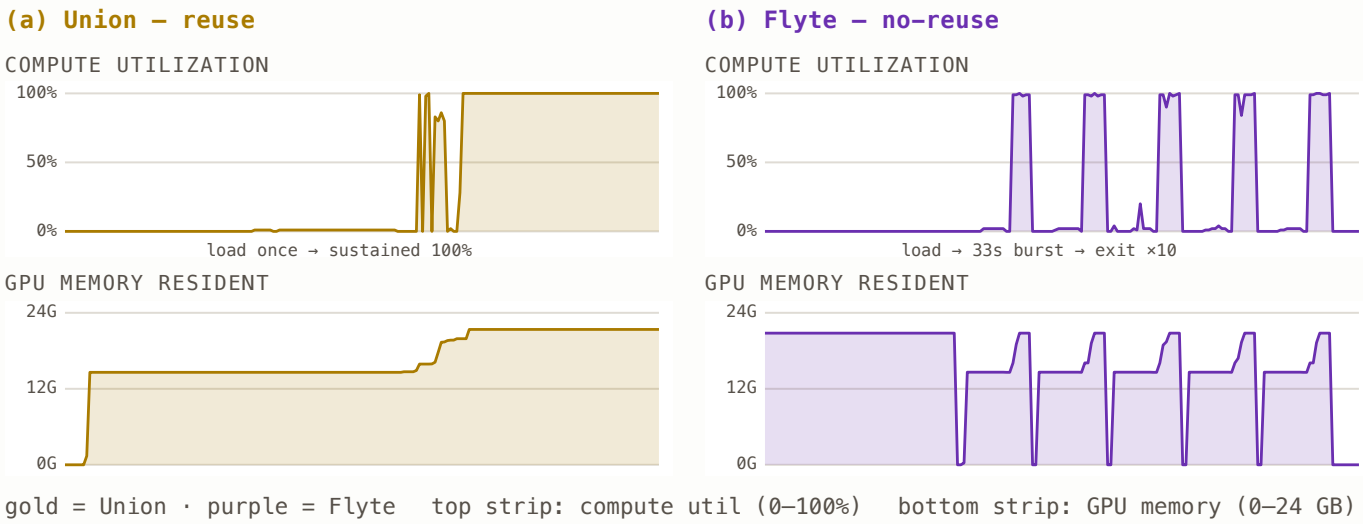

Figure 1. Measured with `nvidia-smi` at 2 s, one L4 per run; compute utilization (top) and GPU memory (bottom) plotted separately. **(a) Union reuse:** one model load, after which utilization holds a steady 100% while the model stays resident. **(b) Flyte no-reuse:** both strips *sawtooth* — ten loads, each a brief 33-s utilization burst, memory rising and falling as pods come and go. Union sampled inside the worker container; Flyte from an on-node DaemonSet.

4. Conclusion

On this workload, container reuse is not an optimization but the difference between a warm, batched accelerator and one that reloads a 7B model ten times to keep it 84% idle. The cheap

resource (scheduling a fresh pod) squanders the expensive one (the GPU). For batch inference of any non-trivial model, keep the worker warm and let a batcher feed it; reserve cold-start-per-call for genuinely one-shot work.

∗ The managed cluster has no node shell, so Union's GPU figures are sampled by running `nvidia-smi` *inside* the reuse worker container and reading it back from the logs; Flyte's come from an on-node DaemonSet. Both are direct measurements, one L4 each.

`runs` Both configurations answer all 500 questions (`ACTION_PHASE_SUCCEEDED`).

`artifact` Flyte v2 · Union reuse run usxwqd4sfhvsgbld7wkl (404 s, in-container GPU trace) · Flyte no-reuse run rwnttmz286vlsbbrwnv5 (1,665 s) · L4 driver 570.195 / CUDA 12.8 · 2026-07.